

Assignment 5

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
    Flatten, Dense, Dropout
from tensorflow.keras.regularizers import l2
import matplotlib.pyplot as plt
import numpy as np
```

```
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
```

```
x_train = x_train.astype('float32') / 255.0
```

```
x_test = x_test.astype('float32') / 255.0
```

```
def create_unidentifiable_model():
```

```
    model = Sequential([
```

```
        Conv2D(16, (3, 3), activation='relu', input_shape=(32, 32, 3))
```

```
        MaxPooling2D((2, 2)),
```

```
        Flatten(),
```

```
        Dense(32, activation='relu'),
```

```
        Dense(10, activation='softmax')
```

```
    ])
    return model
```



```
under_model = create_underfit_model()
```

```
under_model.compile(optimizer='adam',  
                    loss='sparse_categorical_crossentropy',  
                    metrics=['accuracy'])
```

```
under_history = under_model.fit(x_train, y_train,  
                                epochs=15, batch_size=64,  
                                validation_data=(x_test, y_test),  
                                verbose=1)
```

overfitting:

```
def create_overfit_model():
```

```
    model = Sequential([
```

```
        Conv2D(64, (3, 3), activation='relu', input_shape=(32, 32, 3)),
```

```
        Conv2D(128, (3, 3), activation='relu'),
```

```
        Conv2D(256, (3, 3), activation='relu'),
```

```
        MaxPooling2D((2, 2)),
```

```
        Conv2D(512, (3, 3), activation='relu'),
```

```
        Flatten(),
```

```
        Dense(512, activation='relu'),
```

```
        Dense(256, activation='relu'),
```

```
        Dense(128, activation='relu'),
```

```
        Dense(10, activation='softmax')  
    ])
```


return model

over_model = create_overfit_model()

over_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

over_history = over_model.fit(x_train, y_train,
epochs=25, batch_size=64,
validation_data=(x_test, y_test),
verbose=1)

Regularize:

def create_reg_model():

model = Sequential([

Conv2D(64, (3, 3), activation='relu', kernel_regularizer=l2(0.001), input_shape=(32, 32, 3)),

MaxPooling2D((2, 2)),

Conv2D(128, (3, 3), activation='relu', kernel_regularizer=l2(0.001)),

MaxPooling2D((2, 2)),

Flatten(),

Dense(256, activation='relu', kernel_regularizer=l2(0.001)),

Dense(10, activation='softmax')

)


```
return model
```

```
reg_model = create_reg_model = create_reg_model()
```

```
reg_model.compile(optimizer='adam',  
                  loss='sparse_categorical_crossentropy',  
                  metrics=['accuracy'])
```

```
reg_history = reg_model.fit(x_train, y_train,
```

```
epochs=25, batch_size=64, validation_data =  
(x_test, y_test), verbose=1)
```

Dropout

```
def create_dropout_model():
```

```
    model = Sequential([
```

```
        Conv2D(64, (3, 3), activation='relu', input_shape  
              =(32, 32, 3)),
```

```
        MaxPooling2D((2, 2)),
```

```
        Dropout(0.25),
```

```
        Conv2D(128, (3, 3), activation='relu')]
```

```
        MaxPooling2D((2, 2))]
```

```
        Dropout(0.25)
```

```
        Flatten(),
```

```
        Dense(256, activation='relu')]
```


Dropout(0.5),

Dense(10, activation='softmax')

3)

return model

drop-model = create-dropout-model()

drop-model.compile(optimizer='adam', loss='sparse-
categorical_crossentropy', metrics=['accuracy'])

drop-history = drop-model.fit(x_train, y_train, epochs=25,

batch_size=64,

validation_data=(x_test, y_test),

verbose=1)

Combined Model

def create_combined_model():

model = Sequential([

Conv2D(64, (3, 3), activation='relu', kernel-

regularizer=l2(0.001), input_shape=(32, 32, 3))

MaxPooling2D((2, 2)),

Dropout(0.25),

Conv2D(128, (3, 3), activation='relu', kernel-


```

regularizers = L2(0.001)),
Max Pooling 2D ((2, 2)),
Dropout(0.25),
Flatten()
Dense(256, activation = 'relu', kernel_regularizer
= L2(0.001)),
Dropout(0.5),
Dense(10, activation = 'softmax')

```

1)

```

return model

combined_model = create_combined_model()
combined_model.compile(optimizer = 'adam',
                        loss = 'sparse_categorical_crossentropy',
                        metrics = ['accuracy'])

combined_history = combined_model.fit(x_train,
                                       y_train, epochs = 25, batch_size = 64,
                                       validation_data = (x_test, y_test), verbose = 1)

```



```
histories = {  
    "Underfitting": under_history,  
    "Overfitting": over_history,  
    "L2 Regularization": reg_history,  
    "Dropout": drop_history,  
    "Combined": combined_history  
}
```

```
}
```

```
plt.figure(figsize=(14,6))
```

```
for name, hist in histories.items():
```

```
    plt.plot(hist.history['validation accuracy'], label=name)
```

```
plt.title('Validation Accuracy Comparison Across All  
Models:')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Validation Accuracy')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```



```
plt.figure(figsize=(14,6))
```

```
for name, hist in histories.items():
```

```
    plt.plot(hist.history['val_loss'], label=name)
```

```
plt.title('Validation loss comparison')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Validation loss')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

Discussion:

Part A: Underfitting was demonstrated using a very shallow CNN with limited parameters. The model failed to learn complex features from CIFAR-10. In contrast, the overfitting model, which was much deeper with large fully connected layers, achieved very high training accuracy but significantly lower validation accuracy, showing -

clear divergence between training and validation curves after some epochs.

Part B: L2 regularization was applied to the overfitted model with a weight decay of 0.001. The regularization reduced the gap between training and validation accuracy and lowered the final validation loss compared to the baseline overfitted model. This shows that penalizing large weights helps prevent the model from memorizing training data.

Part C: Dropout layers were added to the overfitted model. The model showed improved generalization with higher validation accuracy and more stable training curves. Dropout effectively reduced overfitting by preventing co-adaptation of neurons.

Part-D: Combining L2 regularization and Dropout

produced the best performance among all variants. The model achieved the highest validation accuracy with the smallest gap between training and validation curves. This experiment demonstrated that using regularization and dropout together provides the most effective control over overfitting and leads to better generalization on CIFAR-10.


```

1 import tensorflow as tf
2 from tensorflow.keras.datasets import cifar10
3 from tensorflow.keras.models import Sequential
4 from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
5 from tensorflow.keras.regularizers import l2
6 import matplotlib.pyplot as plt
7 import numpy as np
8
9 # Load and preprocess CIFAR-10
10 (x_train, y_train), (x_test, y_test) = cifar10.load_data()
11 x_train = x_train.astype('float32') / 255.0
12 x_test = x_test.astype('float32') / 255.0
13
14 print("Data loaded:", x_train.shape, x_test.shape)

```

Data loaded: (50000, 32, 32, 3) (10000, 32, 32, 3)

```

1 # Underfitting Model - Very simple architecture
2 def create_underfit_model():
3     model = Sequential([
4         Conv2D(16, (3,3), activation='relu', input_shape=(32,32,3)),
5         MaxPooling2D((2,2)),
6         Flatten(),
7         Dense(32, activation='relu'),
8         Dense(10, activation='softmax')
9     ])
10    return model
11
12 under_model = create_underfit_model()
13 under_model.compile(optimizer='adam',
14                     loss='sparse_categorical_crossentropy',
15                     metrics=['accuracy'])
16
17 print("Training Underfitting Model...")
18 under_history = under_model.fit(x_train, y_train,
19                                epochs=15,
20                                batch_size=64,
21                                validation_data=(x_test, y_test),
22                                verbose=1)

```

Training Underfitting Model...

Epoch 1/15

782/782 ————— 7s 6ms/step - accuracy: 0.3663 - loss: 1.7370 - val_accuracy: 0.4479 - va

Epoch 2/15

782/782 ————— 3s 3ms/step - accuracy: 0.4711 - loss: 1.4651 - val_accuracy: 0.4905 - va

Epoch 3/15

782/782 ————— 3s 3ms/step - accuracy: 0.5101 - loss: 1.3606 - val_accuracy: 0.5179 - va

Epoch 4/15

782/782 ————— 3s 4ms/step - accuracy: 0.5401 - loss: 1.2916 - val_accuracy: 0.5216 - va

Epoch 5/15

782/782 ————— 5s 3ms/step - accuracy: 0.5571 - loss: 1.2429 - val_accuracy: 0.5224 - va

Epoch 6/15

782/782 ————— 3s 3ms/step - accuracy: 0.5744 - loss: 1.2032 - val_accuracy: 0.5578 - va

Epoch 7/15

782/782 ————— 3s 3ms/step - accuracy: 0.5847 - loss: 1.1762 - val_accuracy: 0.5674 - va

Epoch 8/15

782/782 ————— 3s 4ms/step - accuracy: 0.5956 - loss: 1.1443 - val_accuracy: 0.5625 - va

Epoch 9/15

782/782 ————— 3s 3ms/step - accuracy: 0.6066 - loss: 1.1211 - val_accuracy: 0.5764 - va

Epoch 10/15

782/782 ————— 3s 4ms/step - accuracy: 0.6158 - loss: 1.0946 - val_accuracy: 0.5709 - va

Epoch 11/15

782/782 ————— 3s 3ms/step - accuracy: 0.6211 - loss: 1.0765 - val_accuracy: 0.5897 - va

Epoch 12/15

782/782 ————— 3s 3ms/step - accuracy: 0.6289 - loss: 1.0568 - val_accuracy: 0.5749 - va

Epoch 13/15

782/782 ————— 3s 4ms/step - accuracy: 0.6359 - loss: 1.0388 - val_accuracy: 0.5899 - va


```
Epoch 14/15
782/782 ————— 3s 3ms/step - accuracy: 0.6385 - loss: 1.0246 - val_accuracy: 0.5922 - va
Epoch 15/15
782/782 ————— 3s 3ms/step - accuracy: 0.6427 - loss: 1.0167 - val_accuracy: 0.6086 - va
```

```
1 # Overfitting Model - Deep + Large dense layers
2 def create_overfit_model():
3     model = Sequential([
4         Conv2D(64, (3,3), activation='relu', input_shape=(32,32,3)),
5         Conv2D(128, (3,3), activation='relu'),
6         Conv2D(256, (3,3), activation='relu'),
7         MaxPooling2D((2,2)),
8         Conv2D(512, (3,3), activation='relu'),
9         Flatten(),
10        Dense(512, activation='relu'),
11        Dense(256, activation='relu'),
12        Dense(128, activation='relu'),
13        Dense(10, activation='softmax')
14    ])
15    return model
16
17 over_model = create_overfit_model()
18 over_model.compile(optimizer='adam',
19                    loss='sparse_categorical_crossentropy',
20                    metrics=['accuracy'])
21
22 print("Training Overfitting Model...")
23 over_history = over_model.fit(x_train, y_train,
24                               epochs=25,          # Longer training
25                               batch_size=64,
26                               validation_data=(x_test, y_test),
27                               verbose=1)
```

Training Overfitting Model...

```
Epoch 1/25
782/782 ————— 44s 51ms/step - accuracy: 0.4417 - loss: 1.5244 - val_accuracy: 0.5443 -
Epoch 2/25
782/782 ————— 34s 44ms/step - accuracy: 0.6534 - loss: 0.9775 - val_accuracy: 0.6891 -
Epoch 3/25
782/782 ————— 35s 44ms/step - accuracy: 0.7481 - loss: 0.7246 - val_accuracy: 0.7264 -
Epoch 4/25
782/782 ————— 34s 44ms/step - accuracy: 0.8094 - loss: 0.5387 - val_accuracy: 0.7193 -
Epoch 5/25
782/782 ————— 34s 44ms/step - accuracy: 0.8690 - loss: 0.3750 - val_accuracy: 0.7205 -
Epoch 6/25
782/782 ————— 34s 44ms/step - accuracy: 0.9120 - loss: 0.2536 - val_accuracy: 0.7210 -
Epoch 7/25
782/782 ————— 34s 44ms/step - accuracy: 0.9406 - loss: 0.1742 - val_accuracy: 0.7147 -
Epoch 8/25
782/782 ————— 34s 44ms/step - accuracy: 0.9547 - loss: 0.1327 - val_accuracy: 0.7207 -
Epoch 9/25
782/782 ————— 34s 44ms/step - accuracy: 0.9650 - loss: 0.1051 - val_accuracy: 0.7179 -
Epoch 10/25
782/782 ————— 34s 44ms/step - accuracy: 0.9700 - loss: 0.0891 - val_accuracy: 0.7137 -
Epoch 11/25
782/782 ————— 34s 44ms/step - accuracy: 0.9747 - loss: 0.0778 - val_accuracy: 0.7154 -
Epoch 12/25
782/782 ————— 34s 44ms/step - accuracy: 0.9780 - loss: 0.0704 - val_accuracy: 0.7118 -
Epoch 13/25
782/782 ————— 34s 44ms/step - accuracy: 0.9775 - loss: 0.0684 - val_accuracy: 0.7107 -
Epoch 14/25
782/782 ————— 34s 44ms/step - accuracy: 0.9820 - loss: 0.0565 - val_accuracy: 0.7228 -
Epoch 15/25
782/782 ————— 34s 44ms/step - accuracy: 0.9830 - loss: 0.0530 - val_accuracy: 0.7083 -
Epoch 16/25
782/782 ————— 34s 44ms/step - accuracy: 0.9826 - loss: 0.0561 - val_accuracy: 0.6914 -
Epoch 17/25
782/782 ————— 34s 44ms/step - accuracy: 0.9830 - loss: 0.0526 - val_accuracy: 0.7163 -
Epoch 18/25
```



```

782/782 _____ 34s 44ms/step - accuracy: 0.9861 - loss: 0.0454 - val_accuracy: 0.7153 -
Epoch 19/25
782/782 _____ 34s 44ms/step - accuracy: 0.9848 - loss: 0.0493 - val_accuracy: 0.7061 -
Epoch 20/25
782/782 _____ 34s 44ms/step - accuracy: 0.9839 - loss: 0.0499 - val_accuracy: 0.7053 -
Epoch 21/25
782/782 _____ 34s 44ms/step - accuracy: 0.9881 - loss: 0.0382 - val_accuracy: 0.7213 -
Epoch 22/25
782/782 _____ 34s 44ms/step - accuracy: 0.9869 - loss: 0.0437 - val_accuracy: 0.6964 -
Epoch 23/25
782/782 _____ 34s 44ms/step - accuracy: 0.9878 - loss: 0.0397 - val_accuracy: 0.7069 -
Epoch 24/25
782/782 _____ 34s 44ms/step - accuracy: 0.9879 - loss: 0.0393 - val_accuracy: 0.7195 -
Epoch 25/25
782/782 _____ 34s 44ms/step - accuracy: 0.9887 - loss: 0.0360 - val_accuracy: 0.7080 -

```

```

1 # L2 Regularized Model
2 def create_reg_model():
3     model = Sequential([
4         Conv2D(64, (3,3), activation='relu', kernel_regularizer=l2(0.001), input_shape=(32,32,3))
5         MaxPooling2D((2,2)),
6         Conv2D(128, (3,3), activation='relu', kernel_regularizer=l2(0.001)),
7         MaxPooling2D((2,2)),
8         Flatten(),
9         Dense(256, activation='relu', kernel_regularizer=l2(0.001)),
10        Dense(10, activation='softmax')
11    ])
12    return model
13
14 reg_model = create_reg_model()
15 reg_model.compile(optimizer='adam',
16                  loss='sparse_categorical_crossentropy',
17                  metrics=['accuracy'])
18
19 print("Training Regularized (L2) Model...")
20 reg_history = reg_model.fit(x_train, y_train,
21                            epochs=25,
22                            batch_size=64,
23                            validation_data=(x_test, y_test),
24                            verbose=1)

```

Training Regularized (L2) Model...

```

Epoch 1/25
782/782 _____ 11s 10ms/step - accuracy: 0.4636 - loss: 1.6336 - val_accuracy: 0.5555 -
Epoch 2/25
782/782 _____ 5s 6ms/step - accuracy: 0.5841 - loss: 1.3317 - val_accuracy: 0.6104 - va
Epoch 3/25
782/782 _____ 4s 6ms/step - accuracy: 0.6297 - loss: 1.2344 - val_accuracy: 0.6521 - va
Epoch 4/25
782/782 _____ 4s 6ms/step - accuracy: 0.6544 - loss: 1.1787 - val_accuracy: 0.6689 - va
Epoch 5/25
782/782 _____ 5s 6ms/step - accuracy: 0.6754 - loss: 1.1274 - val_accuracy: 0.6724 - va
Epoch 6/25
782/782 _____ 4s 6ms/step - accuracy: 0.6902 - loss: 1.0950 - val_accuracy: 0.6829 - va
Epoch 7/25
782/782 _____ 5s 6ms/step - accuracy: 0.7072 - loss: 1.0590 - val_accuracy: 0.6885 - va
Epoch 8/25
782/782 _____ 5s 6ms/step - accuracy: 0.7201 - loss: 1.0284 - val_accuracy: 0.6849 - va
Epoch 9/25
782/782 _____ 4s 6ms/step - accuracy: 0.7338 - loss: 1.0017 - val_accuracy: 0.7015 - va
Epoch 10/25
782/782 _____ 5s 6ms/step - accuracy: 0.7422 - loss: 0.9867 - val_accuracy: 0.7051 - va
Epoch 11/25
782/782 _____ 4s 6ms/step - accuracy: 0.7496 - loss: 0.9631 - val_accuracy: 0.7206 - va
Epoch 12/25
782/782 _____ 4s 6ms/step - accuracy: 0.7580 - loss: 0.9444 - val_accuracy: 0.7097 - va
Epoch 13/25
782/782 _____ 5s 6ms/step - accuracy: 0.7662 - loss: 0.9279 - val_accuracy: 0.6972 - va
Epoch 14/25
782/782 _____ 4s 6ms/step - accuracy: 0.7752 - loss: 0.9130 - val_accuracy: 0.7136 - va

```



```

Epoch 15/25
782/782 ————— 5s 6ms/step - accuracy: 0.7859 - loss: 0.8931 - val_accuracy: 0.7137 - va
Epoch 16/25
782/782 ————— 5s 6ms/step - accuracy: 0.7885 - loss: 0.8826 - val_accuracy: 0.7202 - va
Epoch 17/25
782/782 ————— 4s 6ms/step - accuracy: 0.7943 - loss: 0.8698 - val_accuracy: 0.7267 - va
Epoch 18/25
782/782 ————— 5s 6ms/step - accuracy: 0.8010 - loss: 0.8583 - val_accuracy: 0.7199 - va
Epoch 19/25
782/782 ————— 5s 6ms/step - accuracy: 0.8041 - loss: 0.8548 - val_accuracy: 0.6857 - va
Epoch 20/25
782/782 ————— 4s 6ms/step - accuracy: 0.8094 - loss: 0.8449 - val_accuracy: 0.7205 - va
Epoch 21/25
782/782 ————— 5s 6ms/step - accuracy: 0.8158 - loss: 0.8333 - val_accuracy: 0.7169 - va
Epoch 22/25
782/782 ————— 4s 6ms/step - accuracy: 0.8202 - loss: 0.8201 - val_accuracy: 0.7134 - va
Epoch 23/25
782/782 ————— 5s 6ms/step - accuracy: 0.8219 - loss: 0.8167 - val_accuracy: 0.7271 - va
Epoch 24/25
782/782 ————— 5s 6ms/step - accuracy: 0.8309 - loss: 0.8065 - val_accuracy: 0.7196 - va
Epoch 25/25
782/782 ————— 4s 6ms/step - accuracy: 0.8321 - loss: 0.8017 - val_accuracy: 0.7194 - va

```

```

1 # Dropout Model
2 def create_dropout_model():
3     model = Sequential([
4         Conv2D(64, (3,3), activation='relu', input_shape=(32,32,3)),
5         MaxPooling2D((2,2)),
6         Dropout(0.25),
7         Conv2D(128, (3,3), activation='relu'),
8         MaxPooling2D((2,2)),
9         Dropout(0.25),
10        Flatten(),
11        Dense(256, activation='relu'),
12        Dropout(0.5),
13        Dense(10, activation='softmax')
14    ])
15    return model
16
17 drop_model = create_dropout_model()
18 drop_model.compile(optimizer='adam',
19                    loss='sparse_categorical_crossentropy',
20                    metrics=['accuracy'])
21
22 print("Training Dropout Model...")
23 drop_history = drop_model.fit(x_train, y_train,
24                               epochs=25,
25                               batch_size=64,
26                               validation_data=(x_test, y_test),
27                               verbose=1)

```

Training Dropout Model...

```

Epoch 1/25
782/782 ————— 15s 13ms/step - accuracy: 0.4198 - loss: 1.5944 - val_accuracy: 0.5716 - va
Epoch 2/25
782/782 ————— 5s 6ms/step - accuracy: 0.5577 - loss: 1.2481 - val_accuracy: 0.6372 - va
Epoch 3/25
782/782 ————— 5s 6ms/step - accuracy: 0.6099 - loss: 1.1122 - val_accuracy: 0.6596 - va
Epoch 4/25
782/782 ————— 5s 6ms/step - accuracy: 0.6402 - loss: 1.0280 - val_accuracy: 0.6873 - va
Epoch 5/25
782/782 ————— 5s 6ms/step - accuracy: 0.6591 - loss: 0.9715 - val_accuracy: 0.6947 - va
Epoch 6/25
782/782 ————— 5s 6ms/step - accuracy: 0.6784 - loss: 0.9190 - val_accuracy: 0.7060 - va
Epoch 7/25
782/782 ————— 5s 6ms/step - accuracy: 0.6901 - loss: 0.8855 - val_accuracy: 0.7175 - va
Epoch 8/25
782/782 ————— 5s 6ms/step - accuracy: 0.7038 - loss: 0.8465 - val_accuracy: 0.7270 - va
Epoch 9/25

```

```

782/782 ————— 5s 6ms/step - accuracy: 0.7141 - loss: 0.8161 - val_accuracy: 0.7233 - va
Epoch 10/25
782/782 ————— 4s 6ms/step - accuracy: 0.7195 - loss: 0.7923 - val_accuracy: 0.7289 - va
Epoch 11/25
782/782 ————— 4s 6ms/step - accuracy: 0.7306 - loss: 0.7548 - val_accuracy: 0.7377 - va
Epoch 12/25
782/782 ————— 5s 6ms/step - accuracy: 0.7384 - loss: 0.7396 - val_accuracy: 0.7357 - va
Epoch 13/25
782/782 ————— 5s 6ms/step - accuracy: 0.7438 - loss: 0.7221 - val_accuracy: 0.7422 - va
Epoch 14/25
782/782 ————— 5s 6ms/step - accuracy: 0.7518 - loss: 0.6976 - val_accuracy: 0.7478 - va
Epoch 15/25
782/782 ————— 5s 6ms/step - accuracy: 0.7601 - loss: 0.6743 - val_accuracy: 0.7328 - va
Epoch 16/25
782/782 ————— 4s 6ms/step - accuracy: 0.7625 - loss: 0.6699 - val_accuracy: 0.7366 - va
Epoch 17/25
782/782 ————— 5s 6ms/step - accuracy: 0.7722 - loss: 0.6423 - val_accuracy: 0.7502 - va
Epoch 18/25
782/782 ————— 4s 6ms/step - accuracy: 0.7735 - loss: 0.6345 - val_accuracy: 0.7499 - va
Epoch 19/25
782/782 ————— 4s 6ms/step - accuracy: 0.7806 - loss: 0.6153 - val_accuracy: 0.7526 - va
Epoch 20/25
782/782 ————— 5s 6ms/step - accuracy: 0.7795 - loss: 0.6133 - val_accuracy: 0.7550 - va
Epoch 21/25
782/782 ————— 4s 6ms/step - accuracy: 0.7871 - loss: 0.5917 - val_accuracy: 0.7517 - va
Epoch 22/25
782/782 ————— 5s 6ms/step - accuracy: 0.7911 - loss: 0.5835 - val_accuracy: 0.7548 - va
Epoch 23/25
782/782 ————— 5s 6ms/step - accuracy: 0.7985 - loss: 0.5661 - val_accuracy: 0.7519 - va
Epoch 24/25
782/782 ————— 4s 6ms/step - accuracy: 0.7970 - loss: 0.5667 - val_accuracy: 0.7606 - va
Epoch 25/25
782/782 ————— 5s 6ms/step - accuracy: 0.8010 - loss: 0.5548 - val_accuracy: 0.7528 - va

```

```

1 # Combined Model
2 def create_combined_model():
3     model = Sequential([
4         Conv2D(64, (3,3), activation='relu', kernel_regularizer=l2(0.001), input_shape=(32,32,3)),
5         MaxPooling2D((2,2)),
6         Dropout(0.25),
7         Conv2D(128, (3,3), activation='relu', kernel_regularizer=l2(0.001)),
8         MaxPooling2D((2,2)),
9         Dropout(0.25),
10        Flatten(),
11        Dense(256, activation='relu', kernel_regularizer=l2(0.001)),
12        Dropout(0.5),
13        Dense(10, activation='softmax')
14    ])
15    return model
16
17 combined_model = create_combined_model()
18 combined_model.compile(optimizer='adam',
19                        loss='sparse_categorical_crossentropy',
20                        metrics=['accuracy'])
21
22 print("Training Combined (L2 + Dropout) Model...")
23 combined_history = combined_model.fit(x_train, y_train,
24                                     epochs=25,
25                                     batch_size=64,
26                                     validation_data=(x_test, y_test),
27                                     verbose=1)

```

Training Combined (L2 + Dropout) Model...

```

Epoch 1/25
782/782 ————— 14s 13ms/step - accuracy: 0.3939 - loss: 1.8170 - val_accuracy: 0.5228 - va
Epoch 2/25
782/782 ————— 5s 6ms/step - accuracy: 0.5068 - loss: 1.5326 - val_accuracy: 0.5877 - va
Epoch 3/25
782/782 ————— 5s 6ms/step - accuracy: 0.5495 - loss: 1.4584 - val_accuracy: 0.6060 - va

```


Epoch 4/25
782/782 ————— 5s 6ms/step - accuracy: 0.5679 - loss: 1.4157 - val_accuracy: 0.6056 - va
Epoch 5/25
782/782 ————— 5s 6ms/step - accuracy: 0.5812 - loss: 1.3837 - val_accuracy: 0.6348 - va
Epoch 6/25
782/782 ————— 5s 6ms/step - accuracy: 0.5943 - loss: 1.3607 - val_accuracy: 0.6366 - va
Epoch 7/25
782/782 ————— 5s 6ms/step - accuracy: 0.6057 - loss: 1.3401 - val_accuracy: 0.6610 - va
Epoch 8/25
782/782 ————— 5s 6ms/step - accuracy: 0.6161 - loss: 1.3225 - val_accuracy: 0.6495 - va
Epoch 9/25
782/782 ————— 5s 6ms/step - accuracy: 0.6221 - loss: 1.3108 - val_accuracy: 0.6809 - va
Epoch 10/25
782/782 ————— 5s 6ms/step - accuracy: 0.6240 - loss: 1.3030 - val_accuracy: 0.6805 - va
Epoch 11/25
782/782 ————— 5s 7ms/step - accuracy: 0.6296 - loss: 1.2896 - val_accuracy: 0.6817 - va
Epoch 12/25
782/782 ————— 5s 6ms/step - accuracy: 0.6367 - loss: 1.2740 - val_accuracy: 0.6881 - va
Epoch 13/25
782/782 ————— 5s 6ms/step - accuracy: 0.6401 - loss: 1.2667 - val_accuracy: 0.6788 - va
Epoch 14/25
782/782 ————— 5s 6ms/step - accuracy: 0.6443 - loss: 1.2625 - val_accuracy: 0.6825 - va
Epoch 15/25
782/782 ————— 5s 6ms/step - accuracy: 0.6471 - loss: 1.2503 - val_accuracy: 0.6869 - va
Epoch 16/25
782/782 ————— 5s 6ms/step - accuracy: 0.6523 - loss: 1.2470 - val_accuracy: 0.6807 - va
Epoch 17/25
782/782 ————— 5s 6ms/step - accuracy: 0.6539 - loss: 1.2392 - val_accuracy: 0.6947 - va
Epoch 18/25
782/782 ————— 5s 6ms/step - accuracy: 0.6591 - loss: 1.2339 - val_accuracy: 0.7010 - va
Epoch 19/25
782/782 ————— 5s 6ms/step - accuracy: 0.6570 - loss: 1.2318 - val_accuracy: 0.6976 - va
Epoch 20/25
782/782 ————— 5s 6ms/step - accuracy: 0.6597 - loss: 1.2293 - val_accuracy: 0.7020 - va
Epoch 21/25
782/782 ————— 5s 6ms/step - accuracy: 0.6654 - loss: 1.2170 - val_accuracy: 0.7084 - va
Epoch 22/25
782/782 ————— 5s 6ms/step - accuracy: 0.6701 - loss: 1.2124 - val_accuracy: 0.7054 - va
Epoch 23/25
782/782 ————— 5s 6ms/step - accuracy: 0.6652 - loss: 1.2197 - val_accuracy: 0.6871 - va
Epoch 24/25
782/782 ————— 5s 6ms/step - accuracy: 0.6689 - loss: 1.2099 - val_accuracy: 0.7114 - va
Epoch 25/25
782/782 ————— 5s 6ms/step - accuracy: 0.6694 - loss: 1.2045 - val_accuracy: 0.7131 - va

```

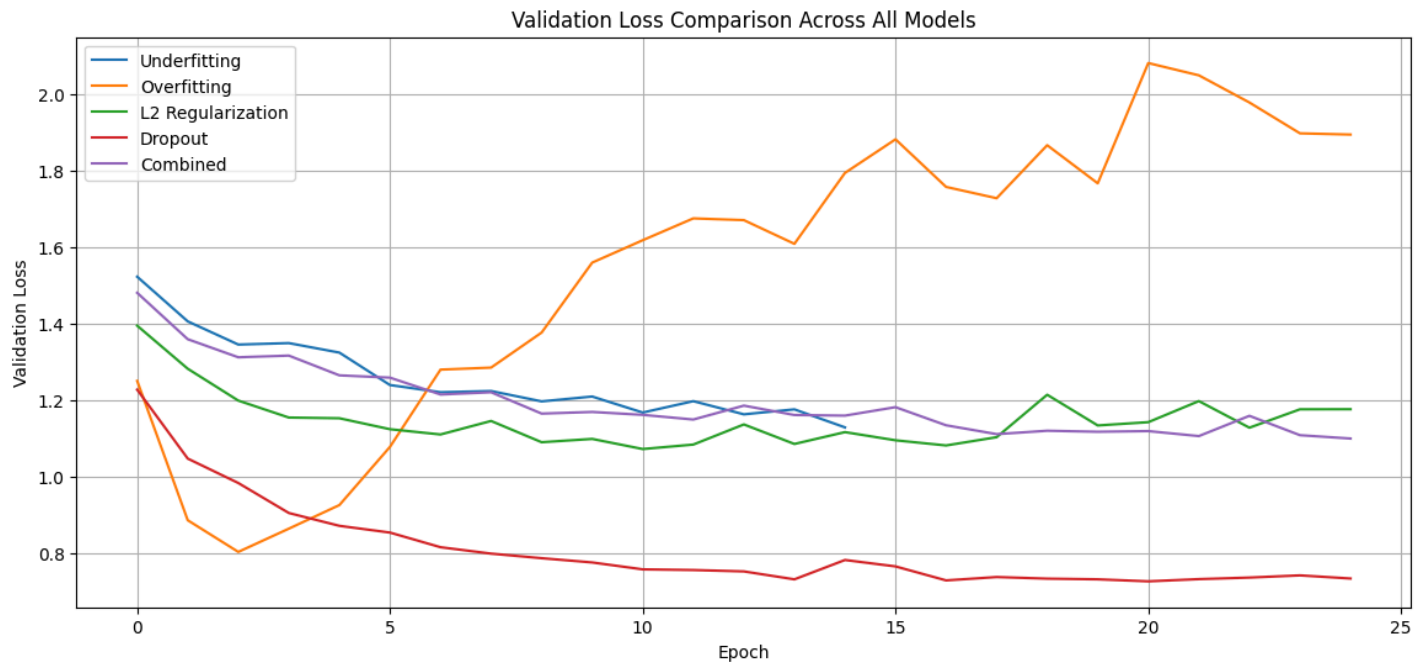
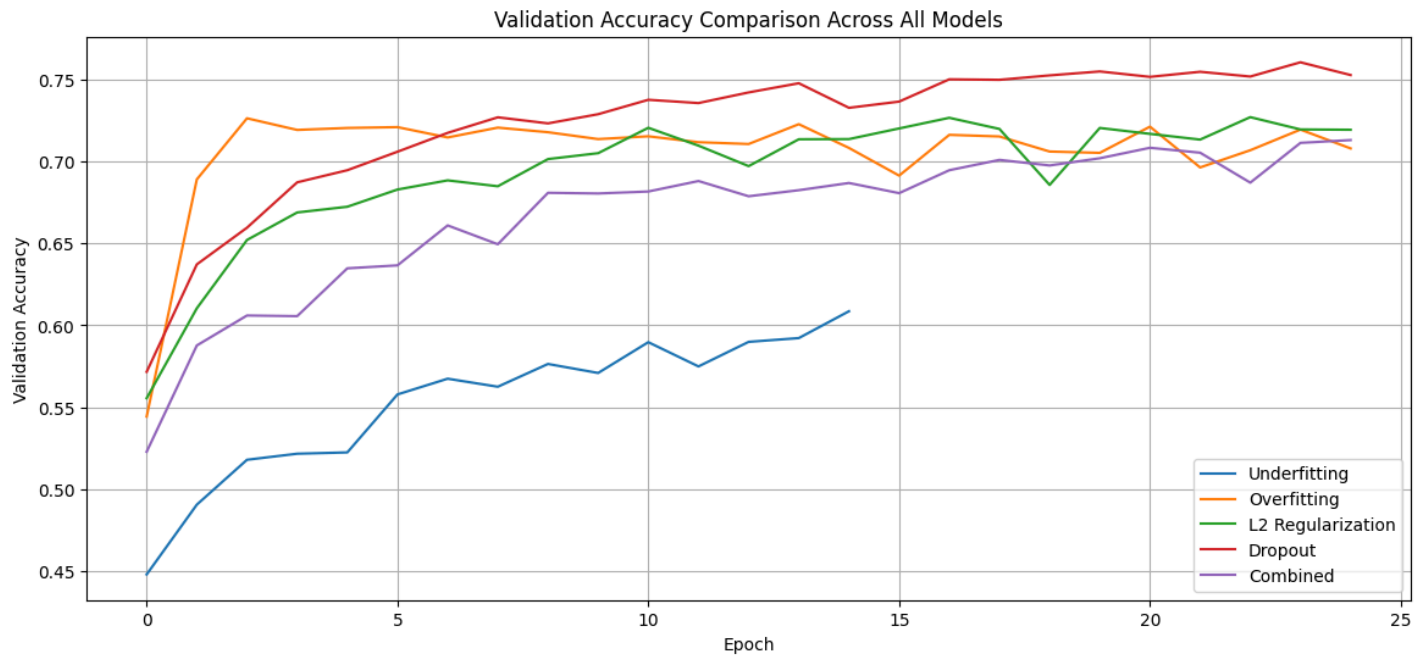
1 histories = {
2     "Underfitting": under_history,
3     "Overfitting": over_history,
4     "L2 Regularization": reg_history,
5     "Dropout": drop_history,
6     "Combined": combined_history
7 }
8
9 # Accuracy Comparison
10 plt.figure(figsize=(14, 6))
11 for name, hist in histories.items():
12     plt.plot(hist.history['val_accuracy'], label=name)
13 plt.title('Validation Accuracy Comparison Across All Models')
14 plt.xlabel('Epoch')
15 plt.ylabel('Validation Accuracy')
16 plt.legend()
17 plt.grid(True)
18 plt.show()
19
20 # Loss Comparison
21 plt.figure(figsize=(14, 6))
22 for name, hist in histories.items():
23     plt.plot(hist.history['val_loss'], label=name)
24 plt.title('Validation Loss Comparison Across All Models')
25 plt.xlabel('Epoch')

```

```

26 plt.ylabel('Validation Loss')
27 plt.legend()
28 plt.grid(True)
29 plt.show()

```



```

1 print("\n=== Final Test Accuracy Comparison ===")
2 for name, model in zip(histories.keys(),
3                        [under_model, over_model, reg_model, drop_model, combined_model]):
4     _, acc = model.evaluate(x_test, y_test, verbose=0)
5     print(f"{name:20s}: {acc:.4f} ({acc*100:.2f}%)")

```

```

=== Final Test Accuracy Comparison ===
Underfitting      : 0.6086 (60.86%)
Overfitting       : 0.7080 (70.80%)
L2 Regularization : 0.7194 (71.94%)
Dropout           : 0.7528 (75.28%)
Combined          : 0.7131 (71.31%)

```